

Calculating Sufficient Reasons for Random Forest Classifiers

Ashlin Iser

Algorithm Engineering, Institute of Theoretical Informatics, KIT-Department of Informatics,
Karlsruhe Institute of Technology (KIT), Germany

Abstract

In this paper, we formalize the implementations of decision tree and random forest classifiers of the Python Scikit-learn package, and we present a CNF encoding which can be used to calculate sufficient reasons a.k.a. prime implicants for them. Our decision tree encoding resembles a monotonic combinational circuit with pure input variables, of which we take advantage in our method for incremental enumeration of its prime implicants. Encoding the combination of several decision trees in a random forest would add a non-monotonic evaluation function to the root of the encoded circuit. In our approach, we solve an auxiliary SAT problem for enumerating valid leaf-node combinations of the random forest. Each valid leaf-node combination is used to incrementally update the original monotonic circuit encoding by extending the DNF at its root with a new term. Preliminary results show that enumeration of prime implicants by incrementally updating the encoding is by order of magnitudes faster than one-shot solving of the monolithic formula. We present preliminary runtime data, and some initial data about the size and number of samples found when translating the prime implicants back into database queries.

1 Introduction

1.1 Motivation

Models created by inductive learning algorithms are useful in practice. The explainability of such automatically generated models is the subject of current research projects [4, 9]. In particular, in applications for which accountability concerns arise, it is imperative that what is learned can be explained [18].

Automatic generation of explanations for machine-learned models requires to formalize those models. The symbolic representation of what is learned facilitates the application of deductive methods for reasoning about what is learned. Deductive reasoning about inductively generated models is usually referred to by the term eXplainable AI (XAI) [3].

The term *explanation* is ambiguous. There is a plethora of possible formalizations leading to different definitions of explainability. Audemard et al. analyze the complexity of different kinds of XAI queries, among which are also prime implicants [5]. One of the explanation queries they analyze is prime implicants, which are also known as sufficient reasons [9]. Choi et al. present an encoding of decision tree and random forest classifiers for computing prime implicants [8]. However, they report on limitations of their approach when encoding cardinality constraints for multivalued variables.

1.2 Contributions

In this paper, we present a propositional, conjunctive normal form (CNF) encoding of decision tree classifiers and random forests. The formula which we encode resembles a *monotonic* combinatorial circuit, which makes it easy to compute prime implicants for it by using an off-the-shelf

incremental SAT solver. We present a tool which can encode decision tree and random forest classifiers generated with the Python package `scikit-learn` [17] into propositional formulas. Our tool uses the SAT solver `CaDiCaL` [7] via its `IPASIR` [6] interface to generate prime implicants. All prime implicants are decoded into the minimal set of case-distinctions in the features space which they represent. We evaluate our tool on classifiers for SAT instances and show that prime implicants deliver concise explanations of classifier decisions.

1.3 Outline

The document is structured as follows. In Section 2, we present a formalization of decision tree classifiers and random forest classifiers as they are implemented in the Python package `scikit-learn`. The section concludes with the required fundamentals on propositional logic, incremental SAT solvers and prime implicants. In Section 3, we present a propositional encoding of the previously formalized classifiers and an algorithm to compute prime implicants for them. We present our implementation in Section 4, where we also evaluate our approach on a classifier which was trained to associate SAT instances to their instance family. We also evaluate our approach on a classifier which was trained to select SAT solvers from small portfolios of solvers drawn from SAT Competition 2020. We conclude with Section 5.

2 Preliminaries

In the following sections, we formally describe the data structures created by *decision tree* (Section 2.1.1) and *random forest* (Section 2.1.2) classifiers which realize the learned prediction function as they are implemented in the Python package `scikit-learn` [17]. In Section 2.2, we introduce some basic notions of propositional logic in particular regarding prime implicants. The formalizations will later serve as a fundament to the propositional encoding of the prediction models under consideration (Section 3).

2.1 Classification

The *classification problem* under consideration is specified as follows. Given a set of training samples $T \subset \mathbb{R}^n$, a set of classes K , and a left-total ground-truth relation $G \subset T \times K$, devise a prediction function $c : \mathbb{R}^n \rightarrow K$ which maximizes the cardinality of correctly classified training samples $|\{t \in T \mid (t, c(t)) \in G\}|$. The challenge is to devise a prediction function which generalizes to yet unseen samples of the feature space. In theory, decision trees can grow until they classify each training sample correctly, i.e., the classification result equals ground truth. In practice however, pruning techniques such as maximum depth or minimum leaf size are used in order to improve the generalization of a classifier to yet unseen samples. A profound introduction to methods and challenges of classification tasks and supervised learning in general can be found in Hastie et al. [11]. The following formalization depends on the concrete implementations `DecisionTreeClassifier` and `RandomForestClassifier` in the Python package `scikit-learn` [17].

2.1.1 Decision Tree Classifiers

Let a classification instance (T, K, G) and its solution in form of a *decision tree* $\mathcal{D}$ be given. A decision tree $\mathcal{D} = (V, E, f, t)$ is specified by a *binary tree* with nodes V and edges $E \subset V \times V$. The nodes are partitioned into *inner nodes* $V^I := \{v \in V \mid \exists x, (v, x) \in E\}$, and *leaf nodes*

$V^L = V \setminus V^I$. The *root node* r is the special inner node with $\nexists x, (x, r) \in E$. The set of edges E is partitioned into *positive edges* E^+ and *negative edges* E^- such that each inner node $v \in V^I$ has exactly one *positive successor* $\text{hi}(v) \in V$ with $(v, \text{hi}(v)) \in E^+$ and exactly one *negative successor* $\text{lo}(v)$ with $(v, \text{lo}(v)) \in E^-$. Associated with each inner node $v \in V^I$ is a feature index $f : V^I \rightarrow \{1, 2, \dots, n\}$ and a threshold $t : V^I \rightarrow \mathbb{R}$.

Given a set of samples $S \subseteq \mathbb{R}^n$, the decision tree $\mathcal{D}$ induces a partitioning $P_S : V \rightarrow 2^S$ of samples over nodes in V . Starting with $P_S(r) = S$ for root node r , the partitioning is recursively defined as in Equation 1.

$$P_S(v) = \begin{cases} S & \text{iff } v \text{ is the root node} \\ \{(s_0, \dots, s_n)^\top \in P_S(x) \mid s_{f(x)} \leq t(x)\} & \text{iff } v = \text{hi}(x) \\ \{(s_0, \dots, s_n)^\top \in P_S(x) \mid s_{f(x)} > t(x)\} & \text{iff } v = \text{lo}(x) \end{cases} \quad (1)$$

The partitioning P_T of the set of training samples T together with ground truth G induces for each node v a probability distribution p_v over classes $k \in K$ as in Equation 2.

$$p_v(k) = \frac{|\{s \in P_T(v) \mid (s, k) \in G\}|}{|P_T(v)|} \quad (2)$$

The decision tree $\mathcal{D}$ specifies a classification function $c(s) : \mathbb{R}^n \rightarrow K$. Given a sample $s \in \mathbb{R}^n$, there exists exactly one leaf node $v \in V^L$ such that $s \in P_{\{s\}}(v)$. The classification result for sample s is the one with the highest probability in that node, i.e., $c(s) = \arg \max_{k \in K} p_v(k)$.

2.1.2 Random Forest Classifiers

In the context of machine learning, *bagging* is the idea to combine several weak learners to one stronger learner by using some kind of voting to determine one collective prediction [11]. A *random forest* $\mathcal{R}^d = \{(T_i, \mathcal{D}_i) \mid 1 \leq i \leq d\}$ combines a set of d decision trees. Each decision tree $\mathcal{D}_i$ is independently trained on randomly selected subsets of the training samples $T_i \subset T$.

Let a classification problem (T, K, G) and a random forest $\mathcal{R}^d = \{(T_i, \mathcal{D}_i) \mid 1 \leq i \leq d\}$ be given. For each decision tree $\mathcal{D}_i = (V_i, E_i, f_i, t_i)$, the training samples T_i induce a probability distribution $p_v(k)$ for nodes $v \in V_i$ (cf. Section 2.1.1). Given a sample $s \in \mathbb{R}^n$, for each decision tree $\mathcal{D}_i$ there exists exactly one leaf node $v_i \in V_i^L$ such that $s \in P_{\{s\}}(v_i)$. The collective class probabilities $p'(k)$ for a sample s are determined by averaging over the class probabilities in each tree as in Equation 3. The classification result for sample s is then given by $c'(s) = \arg \max_{k \in K} p'(k)$.

$$p'(k) = \frac{1}{d} \sum_{i=1}^d p_{v_i}(k) \quad (3)$$

2.2 Prime Implicants

In this document, propositional formulas are given in conjunctive normal form (CNF). A propositional *formula* F is defined over a finite set of Boolean variables V . Each formula is a conjunction of clauses, a clause is a disjunction of literals and a literal is either a variable or its negation. A set of (non-contradictory) literals over V is a *model* M of a formula F , iff its intersection with any clause in F is non-empty. A model is *complete* iff $|M| = |V|$ and *partial*

otherwise. Each model M of a formula F is also an implicant of F , denoted by $M \models F$. An implicant M is a prime implicant of F iff $\nexists M' \subset M, M' \models F$.

Prime implicants for CNF formulas can efficiently be calculated through eager minimization of a model with an incremental SAT solver [15, 12]. If the CNF formula encodes a *monotonic* combinatorial circuit, it is sufficient to minimize the model for the input variables of that circuit, as a complete assignment can later be deduced from the minimized input assignment [12].

3 Approach

In the following we describe CNF encodings for decision tree classifiers (Section 3.1) and random forests (Section 3.2). The encodings resemble monotonic circuits which constrain feature values at their inputs. In Section 3.3, we outline an algorithm to efficiently compute prime implicants for these formulas. In Section 3.3.2, we show how to decode these prime implicants in order to map them to a set of case distinctions in the feature space.

3.1 Encoding Decision Tree Classifiers

Given a classification instance (T, K, G) and a corresponding decision tree $\mathcal{D} = (V, E, f, t)$, we create its propositional encoding as follows.

3.1.1 Variables

We introduce three sets of Boolean variables (α, β , and γ), which together form the set of variables V over which our encoding F is defined. For each class $k \in K$, we introduce a *class variable* $\alpha(k)$ for indicating the classification result. For each inner node $v \in V^I$, we introduce two *node variables* $\beta^+(v)$ and $\beta^-(v)$, with $\beta^+(v)$ denoting its successor $\text{hi}(v)$, and $\beta^-(v)$ denoting its successor $\text{lo}(v)$. For each feature f , we construct the auxiliary set of thresholds σ_f as specified by Equation 4. From σ_f we construct the auxiliary set of threshold intervals τ_f as shown in Equation 5.

$$\sigma_f := \{t(v) \mid f(v) = f, v \in V\} \cup \{-\infty, \infty\} \quad (4)$$

$$\tau_f := \{(t_0, t_1] \mid t_0, t_1 \in \sigma_f \wedge t_0 < t_1 \wedge \nexists t' \in \sigma_f, t_0 < t' < t_1\} \quad (5)$$

For each feature f and each threshold interval $z \in \tau_f$, we introduce the Boolean variable $\gamma(f, z)$. Variables of type $\gamma(f, z)$ indicate whether the respective interval is excluded by the decision tree. This type of value encoding has previously been proposed [8].

3.1.2 Clauses

For each class $k \in K$, we determine the set of leaf nodes $V_k^L \subseteq V^L$ in which the classifier outputs k . Then we encode the *class constraint* for each class k as depicted in Encoding 3.1.

Encoding 3.1 (Class Constraints).

$$\forall k \in K, \alpha(k) \rightarrow \bigvee_{v \in V_k^L} \beta^+(v)$$

For each inner node $v \in V^I$, we encode the *node constraints* as shown in Encoding 3.2.

Encoding 3.2 (Node Constraints).

$$\begin{aligned}
\forall v \in V^I, \beta^+(\text{hi}(v)) &\rightarrow \beta^+(v) \\
\forall v \in V^I, \beta^-(\text{hi}(v)) &\rightarrow \beta^+(v) \\
\forall v \in V^I, \beta^+(\text{lo}(v)) &\rightarrow \beta^-(v) \\
\forall v \in V^I, \beta^-(\text{lo}(v)) &\rightarrow \beta^-(v)
\end{aligned}$$

For each inner node $v \in V^I$, we encode its *interval constraints* as follows. We split the set of threshold variables $\tau_{f(v)}$ and into two auxiliary sets $\tau^+(v)$ and $\tau^-(v)$ which are defined as follows.

$$\begin{aligned}
\tau^+(v) &:= \{(t_0, t_1] \in \tau_{f(v)} \mid t_1 \leq t(v)\} \\
\tau^-(v) &:= \{(t_0, t_1] \in \tau_{f(v)} \mid t_0 > t(v)\}
\end{aligned}$$

Then we encode the *interval constraints* as in Encoding 3.3.

Encoding 3.3 (Interval Constraints).

$$\begin{aligned}
\forall v \in V^I, \bigwedge_{\tau \in \tau^-(v)} \beta^+(v) &\rightarrow \gamma(v, \tau) \\
\forall v \in V^I, \bigwedge_{\tau \in \tau^+(v)} \beta^-(v) &\rightarrow \gamma(v, \tau)
\end{aligned}$$

3.2 Encoding Random Forest Classifiers

Let a random forest $\mathcal{R}^d = \{(T_1, \mathcal{D}_1), \dots, (T_d, \mathcal{D}_d)\}$ be given. Each decision tree $\mathcal{D}_i$ was trained with the classification instance (T_i, K, G) and is given by $\mathcal{D}_i = (V_i, E_i, f_i, t_i)$.

The encoding for each decision tree is similar to the encoding described in Section 3.1. In particular, the node constraints given by Encoding 3.2 are just the same. But as in a random forest, classes are not simply determined by a singular leaf nodes, we need a different encoding for class constraints (cf. Section 3.2.2). Moreover, we need to adapt the interval encoding for reasons which we will explain in the following section.

3.2.1 Encoding Interval Constraints for Random Forests

As thresholds are chosen independently the size of the threshold set $\xi = |\{t(v_i) \mid f(v_i) = f, v_i \in V_i, 1 \leq i \leq d\}|$ can be very large for a given feature f . In preliminary experiments with 100 decision trees ξ could go into the thousands. The number of interval clauses generated when using Encoding 3.3 is in $\mathcal{O}(\xi^2)$. We therefore use a new encoding for interval constraints which has a higher *constant* size overhead in terms of variables as well as clauses but scales with $\mathcal{O}(\xi)$.

For each feature f with intervals τ_f (defined in Equation 5), we introduce two new sets of variables $\epsilon^+(f, z)$ and $\epsilon^-(f, z)$ with $z \in \tau_f$. The interval order of the $z_i \in \tau_f$ also implies an ordering of the variables $\epsilon^+(f, z_i)$ and $\epsilon^-(f, z_i)$. We first encode for each feature f the *connect constraints*, as depicted in Encoding 3.4.

Encoding 3.4 (Interval Connect Constraints).

$$\begin{aligned}
\forall f \in [1, \dots, n], \forall z_i \in \tau_f, \epsilon^-(f, z_i) &\rightarrow \gamma(f, z_i) \\
\forall f \in [1, \dots, n], \forall z_i \in \tau_f, \epsilon^+(f, z_i) &\rightarrow \gamma(f, z_i)
\end{aligned}$$

We further introduce the following *order constraints*.

Encoding 3.5 (Order Constraints).

$$\begin{aligned} \forall f \in [1, \dots, n], z_i \in \tau_f, \epsilon^+(f, z_1) \rightarrow \dots \rightarrow \epsilon^+(f, z_{|\tau_f|}) \\ \forall f \in [1, \dots, n], z_i \in \tau_f, \epsilon^-(f, z_1) \leftarrow \dots \leftarrow \epsilon^-(f, z_{|\tau_f|}) \end{aligned}$$

Then the new *interval constraints* can be encoded for each leaf nodes in the forest as in Encoding 3.6.

Encoding 3.6 (Interval Constraints).

$$\begin{aligned} \forall v \in V^I, \beta^+(v) \rightarrow \epsilon^+(v, t(v)) \\ \forall v \in V^I, \beta^-(v) \rightarrow \epsilon^-(v, t(v)) \end{aligned}$$

3.2.2 Encoding Class Constraints for Random Forests

Recapitulate how classification works in the random forests described in Section 2.1.2. Given a sample s , for each decision tree $\mathcal{D}_i$ with leaf nodes V_i^L , there is exactly one leaf node $v_i \in V_i^L$ such that $s \in P_{\{s\}}(v_i)$. Given the tuple of leaf nodes $(v_1, \dots, v_d)$, i.e., one for each tree, the class of s is determined by the maximum average class probability over those leaf nodes as depicted in Equation 3.

One way to encode this, would be to encode an arithmetic circuit.¹ For the implementation, which we present in this paper, we took a simpler approach. Consider again the tuples of leaf nodes $L := V_1^L \times \dots \times V_d^L$. To each tuple in $t \in L$, we can assign a class in K by determining the maximum average class probability for the leaf nodes. But the size of L grows exponentially with the number of decision trees in the forest. Fortunately, only a fraction of tuples in L is actually a valid combination of leaf nodes, i.e., they also satisfy the node and interval constraints.

Determining Valid Leaf Tuples In order to determine the set of valid tuples $L' \subseteq L$, we solve an *auxiliary SAT instance*. We use the encoding described in the previous section and add *validity constraints* as depicted in Figure 3.7. This ensures that the tuples allow for at least one interval per feature.

Encoding 3.7 (Validity Constraints).

$$\forall f \in [1, \dots, n], \bigvee_{z_i \in \tau_f} \neg \gamma(f, z_i)$$

Moreover, we encode an *exactly-one constraint* for the leaf nodes of each tree. The constraint consists of an at-least-one clause for the leaf nodes of each tree, and a simple pair-wise encoding of an at-most-one constraint. We then enumerate all solutions to variables in $\{\beta^+(v) \mid v \in V_i^L, 1 \leq i \leq d\}$ with a SAT solver to generate the set of valid tuples L' .

¹In this circuit, for each decision tree in the forest, we would encode a list of bit-vectors representing the probabilities of each class, which are encoded to be equivalent to the known class probabilities – dependent on the activated leaf node. For each class, we would further encode an adder circuit to calculate the sum of each classes probabilities over all trees in the forest. On top of that, we would then encode a circuit which ensures that the sum of probabilities of the class to explain is the maximum of all classes.

Connecting Classes to Tuples For each valid tuple in $t_i \in L'$, we introduce a new variable $\delta(t_i)$ and encode the *tuple constraint* as depicted in Encoding 3.8.

Encoding 3.8 (Tuple Constraints).

$$\forall t_i \in L', \bigwedge_{v \in t_i} \delta(t_i) \rightarrow \beta^+(v)$$

For each tuple t_i , we determine its class $k_i \in K$ as shown in Equation 3 and encode the class constraints as depicted in Encoding 3.9.

Encoding 3.9 (Class Constraints).

$$\forall k_i \in K, \alpha(k_i) \rightarrow \bigvee_{t_i \in L'} \delta(t_i)$$

3.3 Explaining Classes with Prime Implicants

Given a decision tree or random forest classifier, and a set of classes to explain $X \subseteq K$, we generate the clauses according to the encoding presented in this section and add the *explanation clause* $\bigvee_{k \in X} \alpha(k)$. In most cases, we are interested in shortest explanations for a single class, or we might want to solve the problem repeatedly once for each class in K .

3.3.1 Minimizing Positive Inputs

The presented encodings of decision trees and random forests resemble a monotonic combinatorial circuit. Thus, a minimal assignment to the input variables induces a minimal assignment to all the variables in the circuit. The circuit is rooted in the explanation clause, and the inputs of the circuit are the interval variables. In fact the clauses encode mostly binary implication chains. All implied variables are implied in their positive polarity. For this reason, we can compute prime implicants by minimizing the number of positively assigned interval variables.

Algorithm 1 outlines the procedure. The algorithm receives as input the formula F and the set of input variables I , i.e., the set of all interval variables. It then returns the set of all minimal assignments P under projection to the input variables. In the outer loop (line 2), we subsequently generate models for F (lines 1 and 15). In the inner loop (line 3), we eagerly minimize those models, by successive addition of clauses (lines 5, 8 and 11) which exclude the current minimum of positively assigned input variables $v \in I$ (line 6). The minimization is done in a strictly eager fashion, as we add assumption literals for already negatively assigned input variables (lines 4, 10, and 12). If the assignment can not be further minimized, we record the prime implicant (lines 13 and 14) and continue to find a new model to minimize (line 15).

3.3.2 Decoding Prime Implicants

We can decode each minimal assignment $p \in P$ into a minimal set of case distinctions in the feature space to explain the classes $k \in X$. For each feature f , we iterate the assignment to interval variables $\gamma(f, z_i)$ according to the ordering induced by the intervals $z_i \in \tau_f$. For each adjacent pair $a = \gamma(f, z_i)$ and $b = \gamma(f, z_{i+1})$, we decode case distinctions D as follows. If a is true in p and b is false in p , we decode $D := f > z_i$. If a is false in p and b is true in p , we decode $D := f \leq z_i$. Otherwise, if both a and b are either true or false, we decode nothing.

Algorithm 1: Incremental Computation of Prime Implicants

Input: CNF Formula: F
Input: Input Variables: I
Output: Prime Implicants: P

```

1 (sat, model)  $\leftarrow$  solve( $F, \emptyset$ )
2 while sat do
3   while sat do
4     assumptions  $\leftarrow \emptyset$ 
5     minim  $\leftarrow \emptyset$ 
6     for  $v \in I$  do
7       if  $v \in \text{model}$  then
8         minim  $\leftarrow \text{minim} \cup \{-v\}$ 
9       else
10        assumptions  $\leftarrow \text{assumptions} \cup \{-v\}$ 
11     $F \leftarrow F \cup \text{minim}$ 
12    (sat, model)  $\leftarrow$  solve( $F, \text{assumptions}$ )
13    if not sat then
14       $P \leftarrow P \cup \{-v \mid v \in \text{minim}\}$ 
15    (sat, model)  $\leftarrow$  solve( $F, \emptyset$ )
16 return  $P$ 

```

4 Evaluation

4.1 Implementation and Dataset

An implementation of our approach can be found on GitHub.² It consists of a Python module written in C++ which computes prime implicants (cf. Section 3.3) using the incremental SAT solver CaDiCaL by Armin Biere [7]. The encoding procedure is implemented in Python for the `DecisionTreeClassifier` and `RandomForestClassifier` implementations in the `scikit-learn` package [17].

4.1.1 Dataset

Our implementation accesses data via our `gbd-tools` package³ which we originally presented in [14]. The three datasets which we use in our evaluations can be obtained at <https://gbd.itl.kit.edu/> and are described in the following. The code which we used to calculate the features described in Sections 4.1.1 and 4.1.1 can be found on GitHub.⁴

Meta Features We collected a set of *meta features* in a database of more than 26k benchmark instances taken mainly from SAT Competitions since 2002 [10]. These meta features include the family of an instance as it can be deduced from the benchmark description documents in the Proceedings of the respective SAT Competition.

²<https://github.com/Udopia/pi-explanations>

³<https://pypi.org/project/gbd-tools/>

⁴<https://github.com/sat-clique/cnftools>

Base Features The features in the *base dataset* do not resemble but share some noteworthy coverage with the features used in [19] and [1].

Amounts This set of features includes the number of clauses, variables, the numbers of clauses of sizes 1 to 9, the number of horn clauses, inverse horn clauses, positive clauses, and negative clauses. The set consists of the following sub groups of features.

Distribution over Horn Clauses For this set of features, we count for each variable its number of occurrences in horn clauses. The counts are represented by five features, their mean, variance, minimum, maximum and entropy. We add another five features for the count of variable occurrences in inverse horn clauses.

Balance of Literal Polarities Here we report on two distributions in terms of their mean, variance, minimum, maximum and entropy. The first distribution captures for each variable the fraction of its number of positive occurrences in clauses divided by the number of negative occurrences in clauses. The second distribution captures for each clause the number of positive literals divided by the number of negative literals.

Distribution of Node Degrees Degree distributions by mean, variance, minimum, maximum and entropy for nodes in the variable interaction graph, nodes in the clause graph, variable nodes in the variable-clause graph, clause nodes in the variable-clause graph.

Gate Features The features in the *gate dataset* were calculated for a recovered hierarchical gate structure which was generated by a gate extraction algorithm for cnf formulas [13, 12].

Amounts This sub group of gate features covers the total number of gates, the number of root variables, the number of input variables, the number of generically recognized gates, the number of monotonically nested gates, the number of non-monotonically nested and-gates, the number of non-monotonically nested or-gates, the number of non-monotonically nested trivial equivalence gates, the number of non-monotonically nested equiv- or xor-gates, as well as the number of non-monotonically nested full gate, i.e., max-term encodings, with more than two inputs.

Distribution over Levels For each gate type for which we recorded their amounts in the previous paragraph, we also determine their levels in the decoded hierarchical gate structure, for each type we report on the distribution of levels by their mean, variance, minimum, maximum and entropy.

4.2 Explaining Classification Results

In the following, we present some initial results for classifiers trained on the given dataset. We used the implementations `DecisionTreeClassifier` and `RandomForestClassifier` in the Python package `scikit-learn`. Both were used with their default settings and a seed of zero. For the `RandomForestClassifier` we additionally reduced the number of decision trees to 2 (default is 100).

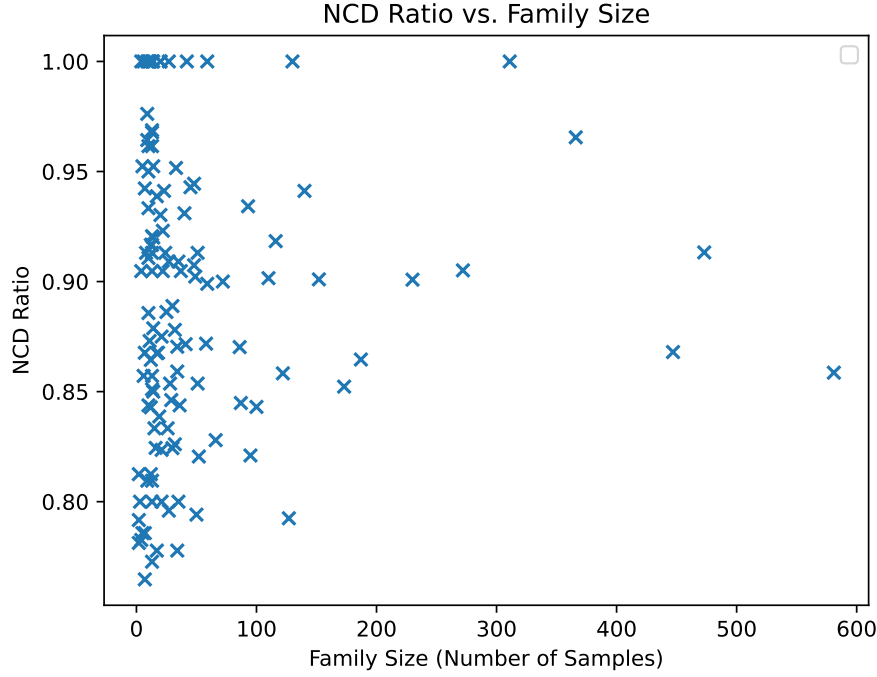


Figure 1: NCD Ratio denotes the fraction of average numbers of case distinctions in prime implicants divided by the average number of case distinctions by leaf nodes. The NCD ratios of all classes are scattered over the number of samples in each class (Family Size)

4.2.1 Instance Family Classifier

We trained a decision tree classifier to predict instance families for instances of SAT Competitions 2002 - 2021. Benchmarks of SAT Competitions 2004 and 2005 were missing. We excluded instances of random instance families and the agile tracks. That left us with a total 6647 instances of 124 distinct instance families. We train a decision tree classifier to predict the family of an instance given the instance features described in Sections 4.1.1 and 4.1.1. The accuracy of the classifier in a cross-validation experiment leaving out 20% of the instances as a testing set is around 94%.

Figure 2 depicts the explanation sizes for four exemplary families in terms of the leaf nodes (blue) and the prime implicants (orange). The x-axis shows the number of case distinctions, which corresponds to the node depths in terms of the leaf nodes and the number of decoded case distinctions in case of the prime implicants. The y-axis shows the number of training samples selected by the respective case distinctions. Using prime implicants we need less case distinctions to explain a classification result, while at the same time covering more samples than with leaf nodes.

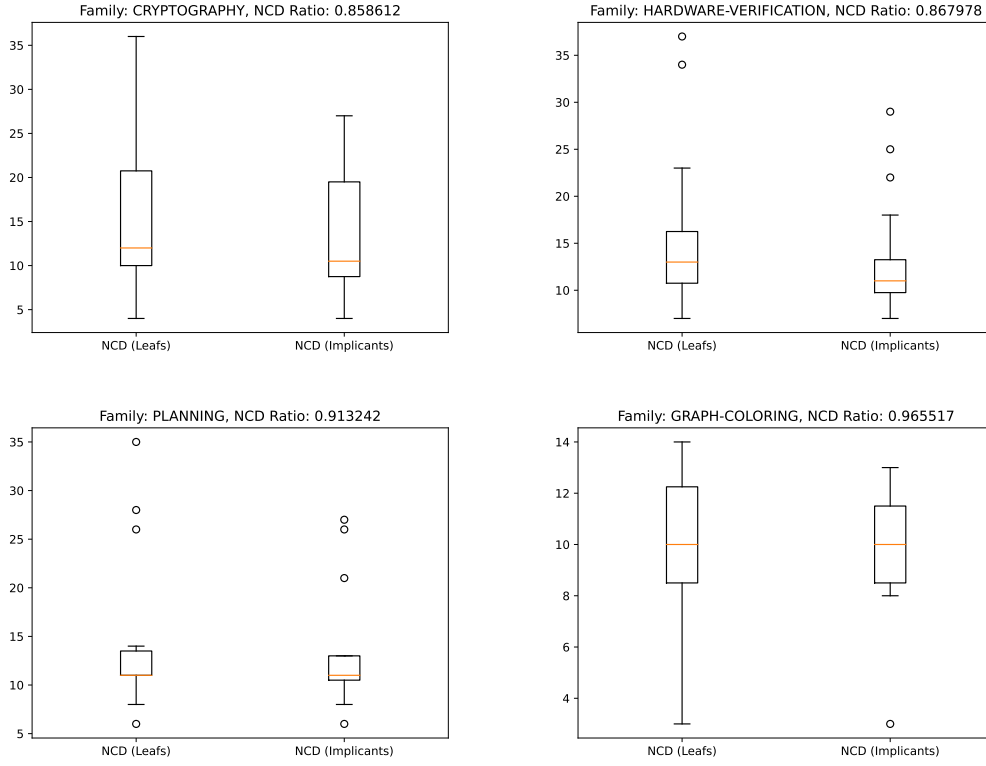


Figure 2: NCD denotes the absolute number of case distinctions in prime implicants (Implicants) and leaf nodes (Leafs). The classifier was trained to separate instances by their family. The plots depict the distributions of these values for the four largest classes *cryptography*, *hardware-verification*, *planning* and *graph-coloring*.

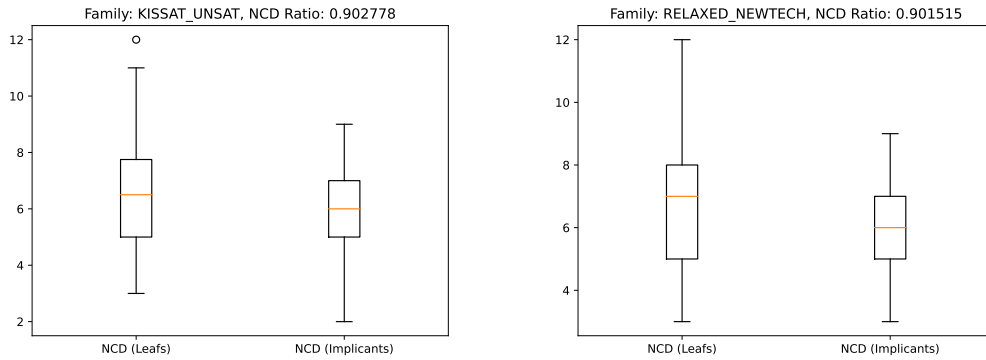


Figure 3: Number of Case Distinctions (NCD) and Number of Covered Training Samples (Cov.) for *Prime Implicants* (Implicants) vs. *Leaf Nodes* (Leafs). The classifier was trained to separate instances for *Kissat Unsat* from instances for *Relaxed NewTech* (by solver speed).

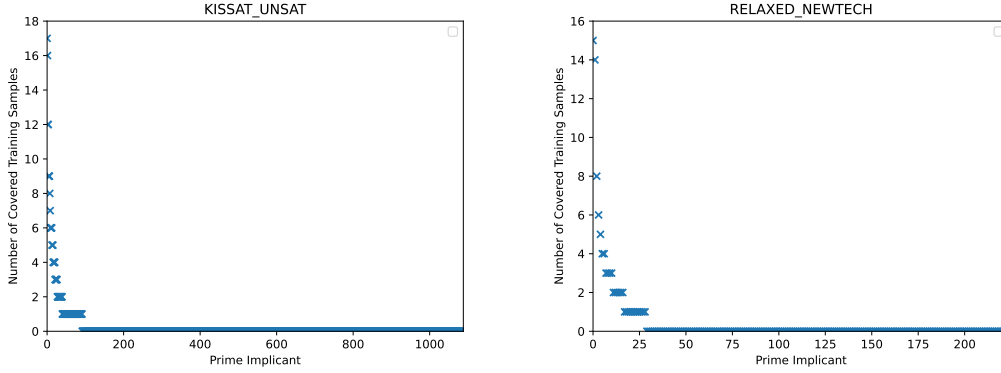


Figure 4: Number of Covered Training Samples for all *Prime Implicants*. The classifier was trained to separate instances for *Kissat Unsat* from instances for *Relaxed NewTech* (by solver speed).

4.2.2 Small Portfolios Classifier

Froleyks et al. report on best small portfolios drawn from solvers of SAT Competition 2020 [10]. The best performing 2-portfolio in terms of its VBS score for the 400 instances in the SAT Competition 2020 benchmarks is *Kissat unsat* and *Relaxed newTech*.

We train a decision tree classifier to predict the fastest solver for an instance given the instance features described in Sections 4.1.1 and 4.1.1. The accuracy of the classifier in a cross-validation experiment leaving out 20% of the instances as a testing set is around 68%. Figure 3 shows that also in this case, with prime implicants we need less case distinctions while at the same time covering more samples when using prime implicants as opposed to leaf nodes.

We also train a random forest classifier consisting of only 2 decision trees for the same data. For this classifier, the accuracy in a cross-validation experiment leaving out 20% of the instances as a testing set raises to around 76%. Figure 4 shows the number of training samples for each prime implicant of the two classes. We observe a large number of prime implicants with zero training samples. This is due to “spurious” valid tuples of leaf nodes emerging in the random forest. The total number of leaf node combinations in this forest is 1518. The number of valid tuples determined by our encoding algorithm is 1312 (cf. Section 3.2.2). The *Kissat unsat* class encompasses a total of 1088 prime implicants. The *Relaxed newTech* class encompasses a total of 224 prime implicants. 92 of those prime implicants encompass training samples of which only 40 cover more than 1 training sample.

We also train a random forest classifier consisting of only 3 decision trees for the same data. For this classifier, the accuracy in a cross-validation experiment leaving out 20% of the instances as a testing set raises to around 78%. In this scenario, we have a total of 66792 leaf node combinations, of which 48880 are valid combinations, i.e., they are reachable by some combination of values in the feature space (cf. Section 3.2.2). The *Kissat unsat* class encompasses a total of 27807 prime implicants of which only 104 prime implicants encompass training samples and only 37 of the prime implicants encompass more than one training sample. The *Relaxed newTech* class encompasses a total of 21073 prime implicants of which only 81 prime implicants encompass training samples and only 31 of the prime implicants encompass more than one training sample.

5 Conclusion

Our evaluation shows that with prime implicants, we indeed need less case distinctions while at the same time covering more samples for explaining classification results. Calculating prime implicants has the advantage that we can extract useful information such as the smallest prime implicant covering most of the samples. This can for example be used to only calculate a small subset of the features for *selecting solver configurations*. Moreover, specific prime implicants can become subject to further research questions, e.g., to reason about the *imposed taxonomy* or for evaluating *biases in benchmarks*.

One problem for our approach is the growing number of interval thresholds in random forests. In initial experiments with the random forest encoding, we see a large number of prime implicants which is not represented by a sample in the training set, i.e., we discover a large number of valid leaf tuples in the random forest which is actually empty (in terms of training samples) but would still produce a classification result for yet unseen samples matching the imposed case distinctions. Our initial implementation of the random forest encoding described in this paper seemed only feasible for random forest of up to 3 decision trees. One reason for this can also be seen in the large number of “spurious” valid leaf *tuples*.

We assume that the number of valid leaf tuples could be reduced by reducing the allowed feature thresholds, e.g., through rounding, in the implementation of the classifier. This would reduce the number of intervals for each feature and could thus positively impact the number of valid nodes due to less implicants with zero training samples. This could in turn also positively impact the performance of our approach and lead to more expressive implicants for random forests. We also intend to implement and analyze the encoding for random forests which we described in the footnote of Section 3.2.

In future work, we want to analyze benchmark taxonomies in greater detail under inclusion of new features, e.g., graph modularity or clustering based features [2, 16]. We will also study feature importance measures and generalization power based on prime implicants. Moreover, we will evaluate portfolio performance with solver selection based on only a few prime implicants.

References

- [1] Enrique Matos Alfonso and Norbert Manthey. New CNF features and formula classification. In Daniel Le Berre, editor, *POS-14. Fifth Pragmatics of SAT workshop, a workshop of the SAT 2014 conference, part of FLoC 2014 during the Vienna Summer of Logic, July 13, 2014, Vienna, Austria*, volume 27 of *EPiC Series in Computing*, pages 57–71. EasyChair, 2014.
- [2] Carlos Ansótegui, Maria Luisa Bonet, Jesús Giráldez-Cru, Jordi Levy, and Laurent Simon. Community structure in industrial SAT instances. *J. Artif. Intell. Res.*, 66:443–472, 2019.
- [3] Alejandro Barredo Arrieta, Natalia Díaz Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-Lopez, Daniel Molina, Richard Benjamins, Raja Chatila, and Francisco Herrera. Explainable artificial intelligence (XAI): concepts, taxonomies, opportunities and challenges toward responsible AI. *CoRR*, abs/1910.10045, 2019.
- [4] Gilles Audemard, Steve Bellart, Louenas Bounia, Frédéric Koriche, Jean-Marie Lagniez, and Pierre Marquis. On the computational intelligibility of boolean classifiers. In Meghyn Bienvenu, Gerhard Lakemeyer, and Esra Erdem, editors, *Proceedings of the 18th International Conference on Principles of Knowledge Representation and Reasoning, KR 2021, Online event, November 3-12, 2021*, pages 74–86, 2021.
- [5] Gilles Audemard, Frédéric Koriche, and Pierre Marquis. On tractable XAI queries based on compiled representations. In Diego Calvanese, Esra Erdem, and Michael Thielscher, editors, *Pro-*

- ceedings of the 17th International Conference on Principles of Knowledge Representation and Reasoning, KR 2020, Rhodes, Greece, September 12-18, 2020*, pages 838–849, 2020.
- [6] Tomás Balyo, Armin Biere, Ashlin Iser, and Carsten Sinz. SAT race 2015. *Artif. Intell.*, 241:45–65, 2016.
 - [7] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximillian Heisinger. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020. In Tomas Balyo, Nils Froleys, Marijn Heule, Ashlin Iser, Matti Järvisalo, and Martin Suda, editors, *Proc. of SAT Competition 2020 – Solver and Benchmark Descriptions*, volume B-2020-1 of *Department of Computer Science Report Series B*, pages 51–53. University of Helsinki, 2020.
 - [8] Arthur Choi, Andy Shih, Anchal Goyanka, and Adnan Darwiche. On symbolically encoding the behavior of random forests. *CoRR*, abs/2007.01493, 2020.
 - [9] Adnan Darwiche and Auguste Hirth. On the reasons behind decisions. In Giuseppe De Giacomo, Alejandro Catalá, Bistra Dilkina, Michela Milano, Senén Barro, Alberto Bugarín, and Jérôme Lang, editors, *ECAI 2020 - 24th European Conference on Artificial Intelligence, 29 August-8 September 2020, Santiago de Compostela, Spain, August 29 - September 8, 2020 - Including 10th Conference on Prestigious Applications of Artificial Intelligence (PAIS 2020)*, volume 325 of *Frontiers in Artificial Intelligence and Applications*, pages 712–720. IOS Press, 2020.
 - [10] Nils Froleys, Marijn Heule, Ashlin Iser, Matti Järvisalo, and Martin Suda. SAT competition 2020. *Artif. Intell.*, 301:103572, 2021.
 - [11] Trevor Hastie, Robert Tibshirani, and Jerome H. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd Edition*. Springer Series in Statistics. Springer, 2009.
 - [12] Ashlin Iser. *Recognition and Exploitation of Gate Structure in SAT Solving*. PhD thesis, Karlsruhe Institute of Technology, Germany, 2020.
 - [13] Ashlin Iser, Norbert Manthey, and Carsten Sinz. Recognition of nested gates in CNF formulas. In Marijn Heule and Sean A. Weaver, editors, *Theory and Applications of Satisfiability Testing - SAT 2015 - 18th International Conference, Austin, TX, USA, September 24-27, 2015, Proceedings*, volume 9340 of *Lecture Notes in Computer Science*, pages 255–271. Springer, 2015.
 - [14] Ashlin Iser and Carsten Sinz. A problem meta-data library for research in SAT. In Daniel Le Berre and Matti Järvisalo, editors, *Proceedings of Pragmatics of SAT 2015, Austin, Texas, USA, September 23, 2015 / Pragmatics of SAT 2018, Oxford, UK, July 7, 2018*, volume 59 of *EPiC Series in Computing*, pages 144–152. EasyChair, 2018.
 - [15] Ashlin Iser, Carsten Sinz, and Mana Taghdiri. Minimizing models for tseitin-encoded SAT instances. In Matti Järvisalo and Allen Van Gelder, editors, *Theory and Applications of Satisfiability Testing - SAT 2013 - 16th International Conference, Helsinki, Finland, July 8-12, 2013. Proceedings*, volume 7962 of *Lecture Notes in Computer Science*, pages 224–232. Springer, 2013.
 - [16] Chunxiao Li, Jonathan Chung, Soham Mukherjee, Marc Vinyals, Noah Fleming, Antonina Kolokolova, Alice Mu, and Vijay Ganesh. On the hierarchical community structure of practical boolean formulas. In Chu-Min Li and Felip Manyà, editors, *Theory and Applications of Satisfiability Testing - SAT 2021 - 24th International Conference, Barcelona, Spain, July 5-9, 2021, Proceedings*, volume 12831 of *Lecture Notes in Computer Science*, pages 359–376. Springer, 2021.
 - [17] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of machine learning research*, 12(Oct):2825–2830, 2011.
 - [18] Chris Percy, Simo Dragicevic, Sanjoy Sarkar, and Artur S. d’Avila Garcez. Accountability in AI: from principles to industry-specific accreditation. *CoRR*, abs/2110.09232, 2021.
 - [19] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Satzilla: Portfolio-based algorithm selection for SAT. *J. Artif. Intell. Res.*, 32:565–606, 2008.